Forms and user input

Forms are how users send data to a server. Signups, logins, search boxes, file uploads, etc all use forms in one way or another. Underneath the variety, a form does one simple thing: it collects a list of name and value pairs and sends them somewhere.

Forms are the most interactive part of plain HTML, and one of the few places where the browser, the network, and the server all meet on the same page.

The form element

A form wraps the controls a user fills in. Two attributes set up the trip the data takes.

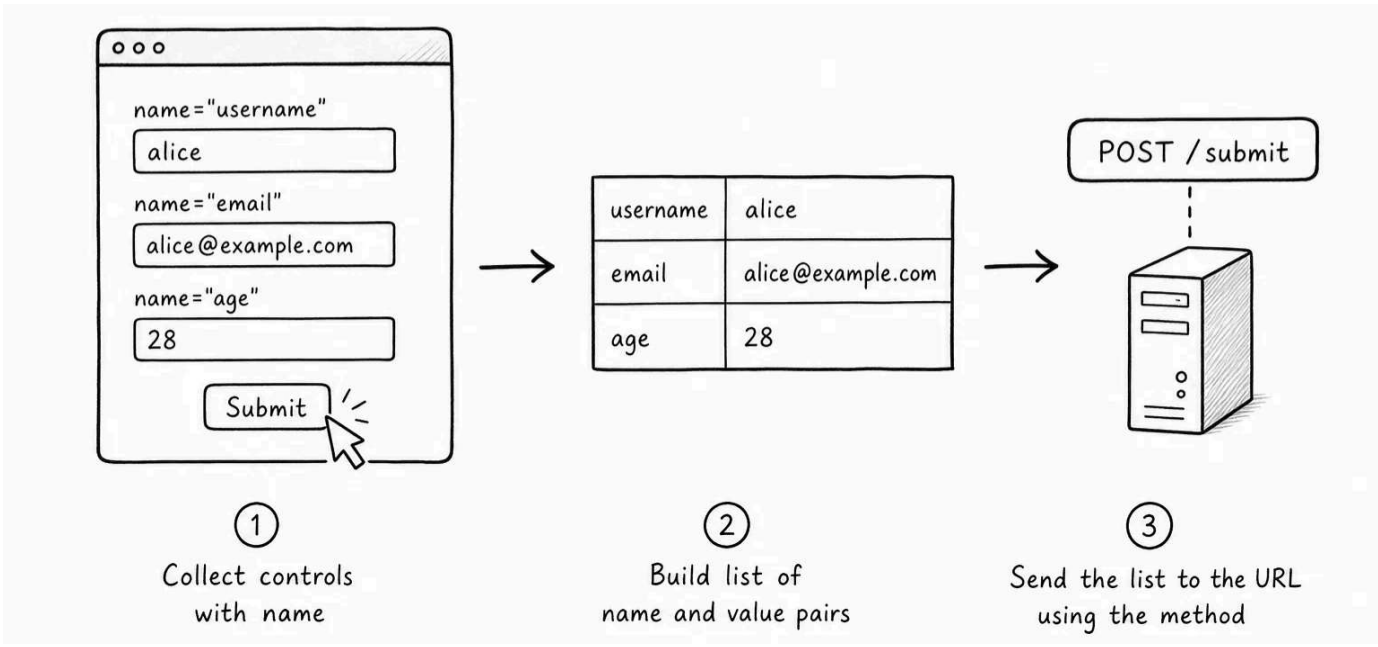
```html
<form action="/signup" method="post">
  ...
</form>
```

- `action` is the URL the form sends its data to. If you don't provide it, the form submits data back to the current page's URL.
- `method` is the HTTP method used. The two normal choices in plain HTML are `get` and `post`.

When the user clicks the submit button, the browser does three things in order:

1. It collects every form control that has a `name` attribute.
2. It builds a list of name and value pairs from those controls.
3. It sends that list to the URL in `action`, using the method in `method`.

That sequence is the whole form lifecycle. Everything else in this lesson is about what each step looks like and how to control it.



GET vs POST: where the data goes

The `method` attribute decides how the form data travels to the server.

A `GET` form puts the data into the URL as a query string:

```text
GET /search?q=html&page=2
```

This is right for things like searches, filters, and pagination. The data is short, not sensitive, and the URL can be bookmarked or shared. Reloading the page just runs the same query again.

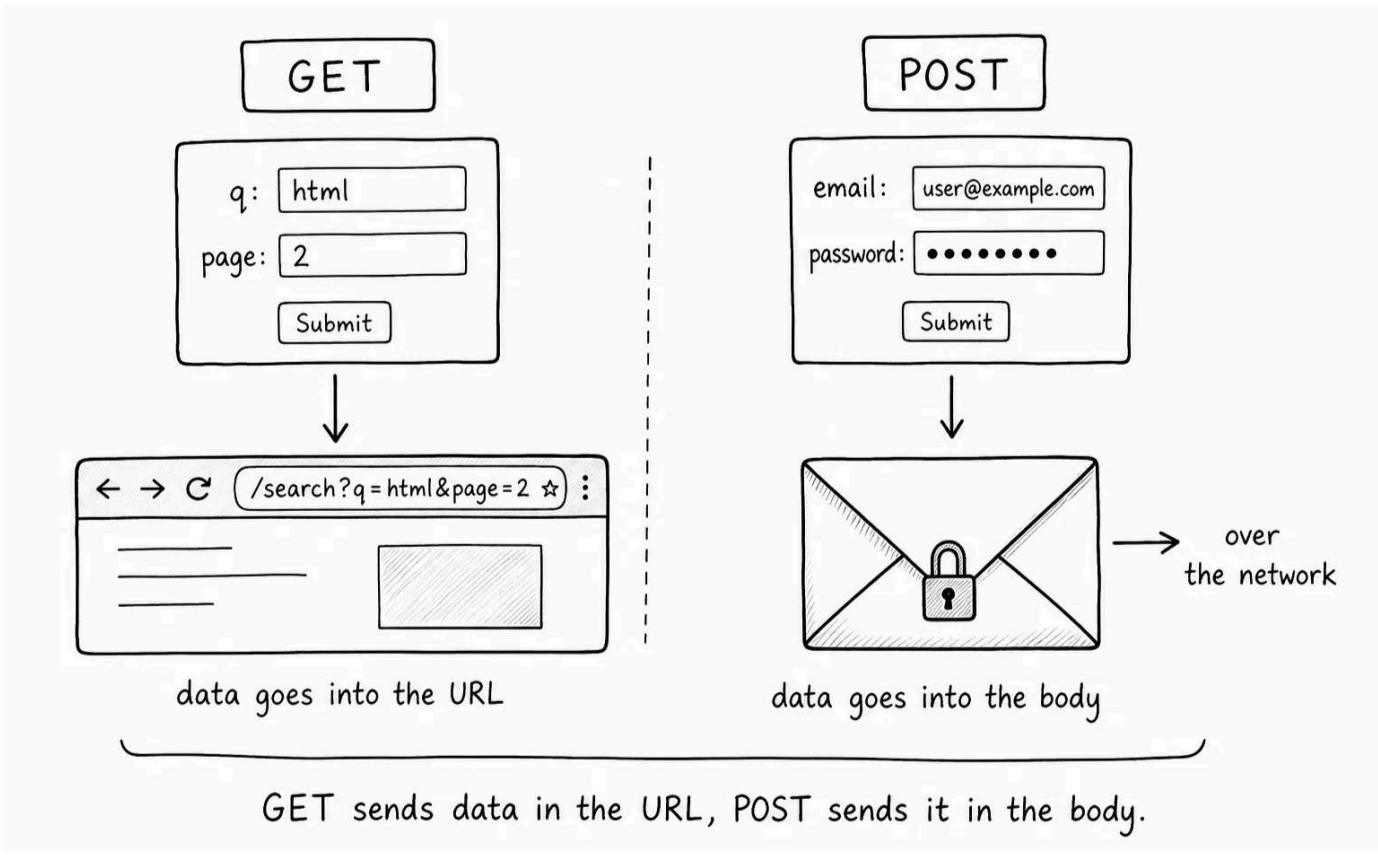
A `POST` form puts the data into the request body instead:

```text
POST /signup
Content-Type: application/x-www-form-urlencoded

email=alice%40example.com&password=secret123
```

This is right for anything that changes data on the server: signups, logins, comments, settings updates, etc. The data is no longer in the URL, which keeps passwords and other private values out of browser history, link previews, and server access logs.

A rough rule of thumb: use `GET` when the form is asking for information, and `POST` when the form is making something happen.



There are other HTTP methods, like `PUT`, `PATCH`, and `DELETE`, but plain HTML forms only natively support `GET` and `POST`. The others are reached through JavaScript or through framework-specific overrides, which you will meet later.

> ⚠ **POST is not encryption** - `POST` keeps form data out of the URL, but it does not encrypt the request. HTTPS is what protects the data while it travels over the network. Any form that sends passwords, payment details, private messages, or personal data should be served over HTTPS.

Inputs, names, and values

The core building block inside a form is the `<input>` element.

```html
<input type="text" name="username" value="">
```

Three attributes do the work.

`type` is the kind of input: `text`, `email`, `password`, `checkbox`, and so on. It controls what UI the browser shows and what data the browser will accept.

`name` is the key the server receives. This attribute is the most important one on any input. If you leave `name` off, the input is not submitted at all. The browser will not include it in the name and value pairs. A missing `name` is one of the most common reasons a form "submits" but the server appears to receive nothing.

`value` is the input's current data. For text inputs, it is the initial text. For checkboxes and radio buttons, it is the value the server gets if the user selects

that option. As the user types or clicks, the live value updates in the DOM.

When the form is submitted, the browser builds pairs that look roughly like this:

```text
username=alice
email=alice@example.com
```

If two inputs share the same `name`, the server receives both values under that key. That is how multiple-choice patterns like grouped checkboxes and multi-select dropdowns work.

Labels and why they pair with inputs

Every input should have a label that tells the user what it is for. Use the `<label>` element with a `for` attribute that points to the input's `id`:

```html
<label for="email">Email address</label>
<input id="email" name="email" type="email">
```

There are two valid ways to write the pairing.

The first is the `for` and `id` pattern shown above. The `for` attribute on the label matches the `id` on the input.

The second is to wrap the input directly inside the label:

```html
<label>
  Email address
  <input type="email" name="email">
</label>
```

Both work. The `for` and `id` pattern gives you more freedom to position the label and input separately with CSS, which is why it is the more common choice in real apps.

Labels matter for three reasons:

- **Click target.** Clicking a label focuses or activates the paired input. That makes small controls like checkboxes much easier to hit on touch screens.
- **Accessibility.** Screen readers announce the label when the input is focused. Without a label, they may only announce the input type, which tells the user nothing.

- **Clarity.** Labeled forms communicate more clearly to every user, even before any styling.

> ⚠ **A placeholder is not a label** - The `placeholder` attribute looks like a label, but it disappears as soon as the user starts typing. Always pair an input with a real `<label>`, and use `placeholder` only for short examples or hints inside the field.

Input types

The `type` attribute changes what kind of value an input collects and how the browser presents it.

Type	Use it for
`text`	Free-form short text
`email`	Email addresses (with light browser validation)
`password`	Sensitive text, hidden as the user types
`number`	Whole or decimal numbers, with up and down arrows
`tel`	Phone numbers
`url`	URLs (with light browser validation)
`date`	A date, with a date picker
`time`	A time of day
`checkbox`	A standalone on/off toggle
`radio`	One choice from a group of options
`file`	A file upload (covered later in this lesson)
`hidden`	Data the user does not see or edit, but the server should still receive

Modern browsers adapt the UI to the type. For example, on a mobile device, `type="email"` shows an email-optimized keyboard, and `type="number"` shows a numeric keypad. Picking the right type is one of the easiest ways to make a form feel right on mobile.

For groups of related radio buttons, give every option the same `name`. The browser then treats the group as a single choice and submits only the selected value:

```html
<fieldset>
  <legend>Choose a plan</legend>
  <label><input type="radio" name="plan" value="starter"> Starter</label>
  <label><input type="radio" name="plan" value="team"> Team</label>
  <label><input type="radio" name="plan" value="business"> Business</label>
</fieldset>
```

The `<fieldset>` element groups related controls, and `<legend>` is the label for the whole group. Screen readers announce the legend when the user enters the fieldset, which gives the choice some context. Use a fieldset whenever a set of inputs only makes sense together: an address block, a pair of date fields, a group of radios.

A `hidden` input is useful for passing along data the user does not need to see or edit, like a record ID or a CSRF token. It is still visible in the DOM and on the network, so it is not for secrets. Use it only for values your server wants back with the form.

Beyond `<input>`: textarea, select, and button

Three more elements complete the form vocabulary.

`<textarea>` is for multi-line text:

```html
<label for="bio">Short bio</label>
<textarea id="bio" name="bio" rows="4"></textarea>
```

Unlike `<input>`, a textarea has content and a closing tag. The initial text goes between the opening and closing tags rather than in a `value` attribute. The `rows` attribute is a starting visible height; the user can usually drag the corner to make it bigger.

`<select>` is a dropdown of choices:

```html
<label for="country">Country</label>
<select id="country" name="country">
  <option value="">Choose a country</option>
  <option value="us">United States</option>
  <option value="ca">Canada</option>
  <option value="uk">United Kingdom</option>
</select>
```

Each `<option>` has its own `value` attribute (what the server receives) and visible text (what the user sees). The first option above has an empty value and acts as a "no selection yet" placeholder. Add `multiple` on the `<select>` to allow more than one selection at once.

`<button>` is the element that submits the form:

```html
<button type="submit">Sign up</button>
```

The `type` on a button matters more than it looks. Inside a `<form>`, `type="submit"` is the default and submits the form when clicked. `type="button"` is a plain button that does nothing on its own and is meant for JavaScript to wire up. `type="reset"` clears the form but is rarely useful in practice.

If you leave `type` off a `<button>` inside a `<form>`, the browser treats it as a submit button. That is convenient but can cause accidental submissions when you only meant a clickable element. Set `type` explicitly whenever a button is doing something other than submitting.

Browser validation: helpful, but not security

HTML has built-in attributes that ask the browser to check the input before it submits the form:

- `required`: the field must not be empty.
- `minlength` / `maxlength`: bounds on the text length.
- `min` / `max`: bounds on numeric or date values.
- `pattern="[A-Za-z0-9]+"`: a regular expression the value must match.
- `type="email"` / `type="url"`: the browser checks that the value looks like that kind of value.

```html
<input type="email" name="email" required minlength="3">
```

When the user tries to submit, the browser checks these constraints, shows an error message next to the first bad field, and blocks the submission until everything passes.

That is a UX feature. It is not security. The user can edit the HTML in DevTools, send the request directly from the terminal, or skip the browser entirely. The server must validate every value again, on its own, before doing anything with it. Treat browser validation as a help to honest users, not as a defense against dishonest ones.

⚠ **Always validate on the server** - Anything you check in the browser, you also have to check on the server. The browser runs in the user's

environment, not yours. A determined user can send any data they want, including data that bypasses every HTML constraint on the page.

File uploads

To let users upload files, use `type="file"`. To make the upload actually work, the form needs a special encoding:

```html
<form action="/upload" method="post" enctype="multipart/form-data">
  <label for="avatar">Profile picture</label>
  <input type="file" id="avatar" name="avatar">
  <button type="submit">Upload</button>
</form>
```

`enctype="multipart/form-data"` tells the browser to send the file as a structured upload rather than as URL-encoded text. Without it, the file's contents are not sent properly. Usually only the file name is sent, which is not what you want.

You can let users pick more than one file with `multiple`, and steer the file picker with `accept`:

```html
<input type="file" name="photos" multiple accept="image/*">
```

The `accept` attribute is a UX hint: it sets the default filter in the file picker. It does not stop the user from sending other file types. As with browser validation, the server has to check the file type, size, and contents on its own.

What the server actually receives

The clearest way to understand a form is to look at the request the browser sends.

A search form with `method="get"` and two inputs:

```html
<form action="/search" method="get">
  <input type="text" name="q">
  <input type="number" name="page" value="1">
  <button type="submit">Search</button>
</form>
```

produces a request like:

```text
```

```
GET /search?q=html&page=2 HTTP/1.1
Host: example.com
```

The same form switched to `method="post"`:

```text
POST /search HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

q=html&page=2
```

The same shape, name and value pairs, moves from the URL to the body. That is the only real difference between the two methods at the HTML level.

If you do not see a `name` attribute on a form control, that control will not appear in either place. Empty input boxes still get submitted (with an empty value) as long as they have a `name`. A label without a `for`, or an input without a `name`, will both submit "successfully" but leave gaps you may not notice until the server complains.

When forms come from app data

Most forms in real apps are wired up to a backend that produces the page and processes the submission. A few habits pay off:

- **Escape user-submitted values when you echo them back.** If you show a comment, name, or search term back on the page, your templating engine should HTML-escape it. Otherwise a value like `<script>alert(1)</script>` becomes a real script tag in someone else's browser. This is the most common cause of cross-site scripting.
- **Use a real form library or framework** for anything beyond a trivial form. Validation rules, error display, repopulating fields after errors, and CSRF tokens are easy to get wrong by hand.
- **Send the right status codes.** A successful `POST` usually returns a redirect to the new page (`303 See Other` or `302 Found`), not a fresh HTML response. This pattern, called Post/Redirect/Get, keeps the back button and the reload button from re-submitting the form.

Try it

Build a small form and watch what the browser actually sends:

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Form practice</title>
</head>
<body>
  <h1>Sign up</h1>

  <form action="/signup" method="post">
    <p>
      <label for="email">Email</label>
      <input id="email" name="email" type="email" required>
    </p>
    <p>
      <label for="password">Password</label>
      <input id="password" name="password" type="password" minlength="8" required>
    </p>
    <p>
      <label>
        <input type="checkbox" name="newsletter" value="yes">
        Send me product updates
      </label>
    </p>
    <p>
      <button type="submit">Create account</button>
    </p>
  </form>
</body>
</html>
```

Open the page, then open DevTools and switch to the **Network** tab.
Tick **Preserve log** so the request stays visible after the page navigates. Fill in the form and submit.

A few things to look at on the resulting request:

- The **method** is `POST` and the path is the form's `action`. Open the request and look at its **Payload** or **Request** tab.
- The payload shows the name and value pairs: `email=...&password=...`. The checkbox only appears if you ticked it. That is the `name` and `value` attributes in action.
- Change `method` to `get` and submit again. The same data now appears in the URL instead of the body.
- Remove the `name` from one of the inputs and submit. That field disappears from the payload entirely, even though the user typed into it.

You will get a network error on submit because `/signup` does not exist on your local file. That is expected. The point is to inspect the request the browser builds. If you want to see a real server's response too, change `action` to `https://httpbin.org/post`. That URL echoes the request back, which is useful when poking at forms.

Test Your Knowledge

What does a form do when submitted?

Click to Reveal the Answer

Already Know that | Didn't Know that | Skip Question

LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson